

Ghost Serialization 1.3.3

Complete technical manual — study and reference (A5 / mobile)

Monorepo `ghost-serializer` · version `1.3.3` · Maven `com.ghostserializer` · compile-time KSP + low-allocation runtime.

How to read this manual

This document is meant to be studied in PART order (I → VIII) or by jumping via the index (blue links in the PDF). Tables summarize; paragraphs and JSON examples explain the why and when to use each piece. If something does not match your project, open the `.kt` file indicated under `build/generated/ksp/` after compiling.

Navigation index

In the PDF: tap blue entries to jump between sections. In the PDF reader also use the Bookmarks/Outline menu (side panel).

PART I — Fundamentals

- I.1 Philosophy and architecture
- I.2 Module map

PART II — Public API (`ghost-api` + `Ghost.kt`)

- II.1 Annotations
- II.2 Full `Ghost.kt` API
- II.3 `GhostSerializer` contract

PART III — KSP compiler (compile-time internals)

- III.1 KSP processor
- III.2 `GhostAnalyzer`
- III.3 Emitters and generated code
- III.4 Registry and sharding

PART IV — Internal runtime

- IV.1 Deserialize/encode pipeline
- IV.2 `GhostJsonFlatReader` + `JsonReaderOptions`
- IV.3 `GhostJsonReader` streaming
- IV.4 Writers and warm buffer
- IV.5 Pools and heuristics
- IV.6 JVM vs iOS

PART V — HTTP adapters

[V.1 Gradle plugin](#)

[V.2 Retrofit](#)

[V.3 Ktor](#)

[V.4 Spring Boot](#)

PART VI — CI, tests, and commands

[VI.1 GitHub Actions CI](#)

[VI.2 Gradle commands](#)

[VI.3 Integration-test](#)

[VI.4 Benchmark](#)

PART VII — Publishing, security, errors

[VII.1 Maven Central](#)

[VII.2 ProGuard](#)

[VII.3 Errors and troubleshooting](#)

[VII.4 Security model](#)

PART VIII — integration-test catalog and generated walkthrough

[VIII.1 Full model catalog](#)

[VIII.2 BenchUserSerializer walkthrough](#)

[VIII.3 Module generated registry](#)

References

[API cheat sheet](#)

[Key repo files](#)

[Glossary](#)

[Factual verification](#)

PART I — Fundamentals

1. Ghost philosophy and promise

What problem it solves

When you use Gson, Moshi, or `kotlinx.serialization` with reflection or runtime metadata, every request pays a cost: finding fields, creating adapters, or walking generic trees. In APIs with thousands of requests per second, that shows up in CPU and allocations.

Ghost inverts the approach: all heavy work happens at compile time, with KSP (Kotlin Symbol Processing). You write a normal data class with one annotation; the compiler generates a `UserSerializer.kt` file with

concrete code, no reflection.

Compile-time vs runtime (mental diagram)

YOUR CODE		BUILD (KSP)		RUNTIME
@GhostSerialization data class User	→	UserSerializer.kt (fixed code)	→	Ghost.deserialize<User>(json) calls UserSerializer directly

At runtime Ghost does not do `Class.getDeclaredFields()`, does not build a dynamic `JsonAdapter`. It only:

1. Looks up the serializer in cache or registry (a generated when).
2. Passes JSON to a byte-by-byte parser (`GhostJsonFlatReader`).
3. The generated serializer reads field by field and builds the object.

Quick comparison with other libraries

Aspect	Gson / Moshi (reflectio	kotlinx.serialization	Ghost
----	----	----	----
When code is generate	Runtime (reflect) or optional KSP	KSP	Always KSP
Dynamic JSON model	Yes	Limited	No (fixed models)
R8 / ProGuard	Broad keep rules	Keep rules	Direct code, little keep
Typical hot path	Adapter map	Encoder/Decoder	Flat reader + monomorphic serializer

The performance promise (in plain language)

- Monomorphic paths: the JIT always sees `UserSerializer.deserialize`, not a generic interface with 200 implementations. That enables inlining.
- Byte-by-byte parser: JSON lives in a `ByteArray`; no intermediate DOM tree of objects is created.
- Per-thread pools: reader and writer are reused between calls on the same thread (`ThreadLocal` on JVM).
- Registry with when: resolving `KClass` → `Serializer` is a compiled comparison chain, not a `HashMap` of strings.

What you should NOT expect from Ghost

- Fully dynamic JSON (`Map<String, Any>` without a type) like Jackson for admin APIs.
- Changing the schema at runtime without recompiling.
- Replacing Jackson in a Spring project where 90% of DTOs are POJOs without `@GhostSerialization`.

Ghost shines when your network models are defined and you want maximum performance on Android, JVM server, or KMP iOS.

Complete minimal example

```
// 1. Model (app or shared module)
@GhostSerialization
```

```
data class User(val id: Int, val name: String)

// 2. After compiling, UserSerializer.kt exists (you do not write it)

// 3. Runtime usage
val json = """"{"id":1,"name":"Ana"}""""
val user = Ghost.deserialize<User>(json)
val again = Ghost.encodeToString(user) // {"id":1,"name":"Ana"}
```

If `Ghost.deserialize` throws `NOT_FOUND`, it almost always means: missing annotation, KSP did not run, or on iOS/Wasm the Ghost Gradle plugin is not applied (or `autoRegistration = false`) and no `addRegistry` call was made.

PART II — Public API

2. Monorepo module map

Module	Published	Role
ghost-api	Yes	Public annotations, zero deps
ghost-compiler	Yes	KSP plugin (KotlinPoet)
ghost-compiler-plugin	Yes	Kotlin compiler plugin: links models to serializers on Kotlin/Native and Kotlin/Wasm
ghost-serialization	Yes KMP	Runtime engine
ghost-gradle-plugin	Yes	Wiring in consumer apps
ghost-retrofit	Yes JVM	ConverterFactory
ghost-ktor	Yes KMP	ContentNegotiation ghost()
ghost-spring-boot-starter	Yes JVM	MVC + WebFlux
ghost-integration-test	No	Compiler tests
ghost-benchmark	No	JVM benchmark
ghost-playground	No	Compose wasm/JVM site (GitHub Pages)

Typical consumer dependency: ghost plugin + ghost-serialization (via plugin) + ghost-compiler (KSP).

What your project needs (explained)

1. Gradle plugin `com.ghostserializer.ghost` — configures KSP, adds correct dependencies for Android, JVM, or KMP.
2. `ghost-api` — annotations only (`@GhostSerialization`, etc.). Minimal weight.
3. `ghost-compiler` — KSP processor; generates `*Serializer.kt` in `build/generated/ksp/`.
4. `ghost-serialization` — runtime: Ghost, parsers, writers, primitives.

You do not ship the compiler in your final app as “business logic”; it only runs at compile time. The APK/AAR contains runtime + generated code.

Modules NOT published to Maven Central

- ghost-integration-test: internal regression suite (2,463 tests; part of ciTestJvm — see section 23).
- ghost-benchmark: Gson/Moshi/Kotlinx vs Ghost comparisons.
- ghost-playground: public interactive lab published to GitHub Pages (see [Playground](https://juanchurtado1991.github.io/ghost-serializer/)).

Integration examples (external repos)

Repo	What it validates
---	---
[ghost-android-test-app](https://github.com/ji	Gson/Moshi/KSer vs Ghost, Retrofit, Ktor
[ghost-spring-boot-test-app](https://github.co	Jackson vs Ghost WebFlux
[ghost-ios-test-app](https://github.com/juanc	XCFramework + GhostBridge + Codable

3. ghost-api — Annotations

Package: com.ghost.serialization.annotations. No external dependencies. Ten public annotations:

Annotation	Target	Role
---	---	---
@GhostSerialization	class	Marks model; discriminator, inferred, name
@GhostName	property	JSON alias
@GhostIgnore	property	Omit from JSON
@GhostEncoder	property	Custom encode function
@GhostDecoder	property	Custom decode function
@GhostFlatten	property	Nested JSON → flat property
@GhostWrap	property	Properties → JSON sub-object
@GhostResilient	class/property	Tolerance for invalid types
@GhostFallback	sealed subclass	Default branch
@GhostSignature	sealed subclass	Signature for inferred polymorphism

@GhostSerialization (required on models)

Without this annotation on the class, KSP ignores the type. It is the “contract” that you want codegen.

```
@GhostSerialization
data class Product(val id: Long, val title: String)
```

Optional parameters:

Parameter	Default	Meaning
---	---	---

discriminator	"type"	JSON field name indicating subclass in sealed class
inferred	false	If true, deduces subclass from present fields (no type field)
name	class name	Registry alias if typeName collides

Sealed with discriminator (most common case):

```
@GhostSerialization(discriminator = "kind")
sealed class Event {
    @GhostSerialization
    data class Click(val x: Int, val y: Int) : Event()
    @GhostSerialization
    data class Scroll(val delta: Int) : Event()
}
```

Input JSON:

```
{"kind": "Click", "x": 10, "y": 20}
```

The generated serializer reads kind, does when, and calls the Click deserializer.

Sealed inferred (APIs without type field):

```
@GhostSerialization(inferred = true)
sealed class SmartEvent { /* subclasses with unique fields */ }
```

If only Click has x and y, and Scroll only delta, generated code checks which fields appear in JSON and picks the subclass. More generated code, but tolerates legacy APIs.

@GhostName — JSON name ≠ Kotlin name

```
@GhostSerialization
data class ApiUser(
    @GhostName("user_id") val id: Int,
    val name: String
)
```

- In Kotlin you use id.
- In JSON the field is called user_id.
- The generated serializer maps one to the other; you do not need Gson's @SerializedName.

@GhostIgnore — local-only field, never in JSON

```
@GhostSerialization
data class Session(
    val token: String,
    @GhostIgnore val cachedAt: Long = System.currentTimeMillis()
)
```

On serialize, cachedAt does not appear. On deserialize, Kotlin's default is used. Useful for derived or UI fields.

@GhostFlatten — nested JSON, flat property

External API sends:

```
{"settings": {"theme": "dark", "lang": "es"}}
```

Your model wants:

```
@GhostSerialization
data class Config(
    @GhostFlatten("settings.theme") val theme: String,
    @GhostFlatten("settings.lang") val lang: String
)
```

Ghost “flattens” the path when reading/writing. Several properties can point to different paths in the same subtree.

@GhostWrap — several properties → one JSON sub-object

Inverse of flatten. If you have `val a: String`, `val b: Int` and want:

```
{"meta": {"a": "x", "b": 1}}
```

```
@GhostWrap("meta") val a: String // inside meta in JSON
```

@GhostEncoder / @GhostDecoder — types Ghost does not know

For Instant, UUID, types from another library, you define static functions:

```
object DateCoder {
    fun decode(epoch: Long): Instant = Instant.ofEpochSecond(epoch)
    fun encode(instant: Instant): Long = instant.epochSecond
}
```

```
@GhostSerialization
data class Log(
    @GhostDecoder(provider = DateCoder::class, function = "decode")
    @GhostEncoder(provider = DateCoder::class, function = "encode")
    val created: Instant
)
```

In JSON created will be a number (epoch). The generated serializer calls `DateCoder.decode` instead of `nextString()`.

@GhostResilient — do not break on a rare enum

If the backend sends `"role": "SUPERADMIN"` and your enum only has `USER`, `ADMIN`:

- Without resilient: exception.
- With `@GhostResilient` on the enum or property: null, default value, or skip depending on what KSP generates.

@GhostFallback — sealed with “catch-all”

```
@GhostSerialization
sealed class Payment {
```

```
@GhostSerialization @GhostFallback
data class Unknown(override val rawType: String) : Payment()
}
```

If the discriminator does not match any known subclass, it deserializes to Unknown instead of failing. Ideal for APIs that add new types without notice.

@GhostSignature — inferred on sealed subclasses

Marks which fields uniquely identify a subclass in inferred polymorphism. The compiler builds a decision tree; without this, inferred on complex sealed types may fail analysis.

PART III — KSP compiler (internals)

4. KSP — Compiler entry point

KSP (Kotlin Symbol Processing) runs during compilation, in rounds. Ghost registers its processor via SPI:

META-INF/services/com.google.devtools.ksp.processing.SymbolProcessorProvider → class GhostSerializationProvider.

What happens when you run ./gradlew build

Step 1 — Discovery: KSP finds all classes annotated with @GhostSerialization in the module.

Step 2 — Analysis (GhostAnalyzer): for each class it validates types, names, flatten, sealed, defaults. If something is invalid, compile error with a message (no silent failure in 1.1.17).

Step 3 — Generation (GhostCodeGenerator + emitters): writes to disk, for example:

build/generated/ksp/<variant>/kotlin/<package>/UserSerializer.kt

Step 4 — Global registry: if there was at least one class, it also generates:

- GhostModuleRegistry_my_app.kt — clazz → serializer map
- META-INF/services/com.ghost.serialization.contract.GhostRegistry — for ServiceLoader on JVM
- META-INF/proguard/ghost-serialization.pro — R8 rules

Step 5 — Normal Kotlin compilation: the Kotlin compiler compiles your sources and generated ones together.

Consumer configuration

```
plugins {
    id("com.ghostserializer.ghost") version "1.3.3"
```

```

}

// Optional but recommended with several modules containing models:
ksp {
    arg("ghost.moduleName", "my_app") // → GhostModuleRegistry_my_app
}

```

If you omit `moduleName`, the registry is called `GhostModuleRegistry_Default`. In tests (`src/test`) a `_Test` suffix is added to avoid clashing with production.

Where to look at generated code

Platform	Typical path under <code>build/generated/ksp/</code>
---	---
Android	<code>build/generated/ksp/androidDebug/kotlin/</code>
JVM	<code>build/generated/ksp/main/kotlin/</code>
KMP	<code>build/generated/ksp/metadata/commonMain/kotlin/</code>

Open a `*Serializer.kt` and compare it with PART VIII of this manual (BenchUser walkthrough).

5. GhostAnalyzer and property models

`GhostAnalyzer.kt` is the compiler “notary”: it turns each `KSClassDeclaration` into a list of `GhostPropertyModel` that emitters consume.

Validations it performs (and why they matter)

Validation	If it fails
---	---
Only data class, sealed, enum, value class	Does not generate code for bare interfaces
Supported types (primitives, String, List, Map nested <code>@GhostSerialization</code>)	Error: cannot serialize <code>Flow<T></code> or raw types
Coherent <code>@GhostFlatten</code> / <code>@GhostWrap</code> paths	Avoids JSON impossible to read/write
Unique JSON names per class	Two fields cannot map to the same key
Sealed: subclasses also annotated	Without annotation on child, not in registry
Custom coder: correct function signature	decode must be static and types compatible

What `GhostPropertyModel` stores (per-field metadata)

Each data class property is summarized in an internal object with:

- `kotlinName` / `jsonName` — may differ via `@GhostName`
- `nullable`, `hasDefault`, default value for multi-branch
- `bitIndex` — position in bit mask (field id → bit 0, name → bit 1, ...)
- `flags`: resilient, flatten path, wrap path, uses external encoder/decoder
- `resolved type`: enum, sealed, List of X, etc.

That metadata is what `DeserializeCodeEmitter` uses to write when

(selectNameAndConsume) and constructor branches.

Compile-time errors (1.1.17 behavior)

If the analyzer detects a problem, KSP reports an error and the build fails. It does not generate an empty serializer or “best effort”. This protects production: if it compiles, the model was coherent at compile-time.

6. Code generation — Emitters

Class	Role
GhostCodeGenerator	Orchestrates serialize + deserialize
SerializeCodeEmitter	Serialize body (flat + streaming)
DeserializeCodeEmitter	Deserialize body
StandardEmitter	Normal models
FragmentedEmitter	Huge models (>40 fields)
FragmentedSerializeEmitter	Avoids JVM method > 64KB

Constants: GhostEmitterConstants.kt — %L, %T templates, registry names, chunk size 500 for registry when sharding.

Generate serialize pattern (conceptual)

```
// Generated – simplified idea
object UserSerializer : AbstractGhostSerializer<User>() {
    override val typeName = "User"
    override fun serialize(writer: GhostJsonWriter, value: User) {
        writer.beginObject()
        writer.writeNameHeader(HEADERS[0]) // "id":
        writer.writeInt(value.id)
        writer.writeNameHeader(HEADERS[1]) // "name":
        writer.writeString(value.name)
        writer.endObject()
    }
    override fun deserialize(reader: GhostJsonFlatReader): User {
        reader.beginObject()
        var mask = 0L
        var id = 0
        var name: String? = null
        loop@ while (reader.hasNext()) {
            when (reader.selectNameAndConsume(OPTIONS)) {
                0 -> { id = reader.nextInt(); mask = mask or 1L }
                1 -> { name = reader.nextString(); mask = mask or 2L }
                else -> reader.skipValue()
            }
        }
        reader.endObject()
        if ((mask and REQUIRED) != REQUIRED) throw GhostJsonException("Required field missing")
    }
}
```

```

        return User(id, name!!)
    }
}

```

selectNameAndConsume — O(1)

JsonReaderOptions precomputes a trie of field names. At runtime there is no `== "fieldName"` string compare in the main loop hot path.

Bitmask for required fields

Imagine User with required fields id, name, email. The generator assigns:

- id → bit 1 (value 1)
- name → bit 2 (value 2)
- email → bit 4 (value 4)
- REQUIRED_MASK = 7 (1+2+4)

At the end of the loop, if `(mask and 7) != 7`, some field is missing and `GhostJsonException` is thrown with message "Required field 'email' missing". A single Long checks all required fields; cheaper than three if `(name == null)`.

FragmentedEmitter (>40 fields)

JVM limits bytecode size per method (~64 KB). A `GodObject` with 80 fields would produce a huge deserialize. `FragmentedEmitter` splits the method into several `deserializePart1`, `deserializePart2`, etc., transparent to callers.

7. Registry and discovery

Registry sharding (internal)

If a module has more than 500 models (`REGISTRY_CHUNK_SIZE`), the compiler generates:

- Several methods `getShardMap0()`, `getShardMap1()`, ...
- `getSerializer(clazz)` as a fragmented when to avoid exceeding 64 KB JVM bytecode per method
- lazy `allSerializers` for `prewarm()`

Generated file:
`com.ghost.serialization.generated.GhostModuleRegistry_<moduleName>`
 GhostRegistry interface:

```

interface GhostRegistry {
    fun <T : Any> getSerializer(clazz: KClass<T>): GhostSerializer<T>?
    fun getAllSerializers(): Map<KClass<*>, GhostSerializer<*>>
}

```

Generated: `GhostModuleRegistry_my_app` with `when (clazz)` possibly

sharded if there are thousands of models (>500 per method).

JVM/Android discovery:

```
// Ghost.jvm.kt - idea
fun discoverRegistries(): Iterable<GhostRegistry> {
    // Class.forName fast-path + ServiceLoader
}
```

Generated

file:

META-INF/services/com.ghost.serialization.contract.GhostRegistry

Android/JVM: with ServiceLoader + generated META-INF, you usually do not need addRegistry except for tests or extra manual registries.

Kotlin/Native and Kotlin/Wasm: automatic registration (1.3.2+)

Native and Wasm have no ServiceLoader and no reflection to find a class by name, so discoverRegistries() returns an empty list there. Since 1.3.2 the Gradle plugin closes that gap with associated objects, the same mechanism kotlinx.serialization uses on these targets:

1. com.ghostserializer.ghost applies ghost-compiler-plugin to every Kotlin/Native and Kotlin/Wasm compilation (never to JVM/Android, whose bytecode is unchanged).
2. The compiler plugin adds @GhostSerializerLink(UserSerializer::class) to every @GhostSerialization / @GhostProtoSerialization class (and to unannotated direct subclasses of an annotated sealed type, which map to the parent serializer, as in the generated registry). On these targets GhostSerializerLink is an @AssociatedObjectKey.
3. On a cache miss, after the registries, Ghost.getSerializer calls findAssociatedObject<GhostSerializerLink>() and caches the result in serializerCache. It runs once per class and never on the hot path.

The link lives on the class itself, so it also works when the models are in another Gradle module or klib. No startup code is needed: Ghost.deserialize<User>(json) works on iOS and Wasm right away.

Still needs addRegistry on Native/Wasm: name-based and bulk APIs (getSerializerByName, getSerializerNames, getAllSerializers, and prewarm() warming serializers ahead of time), because associated objects can only be looked up by KClass. A registry added with addRegistry takes precedence over the link.

Opt out / manual fallback:

```
ghost {
    autoRegistration.set(false)
}
```

```
// then, at framework startup:
fun initGhostRuntime() {
    Ghost.addRegistry(GhostModuleRegistry_my_app.INSTANCE)
    Ghost.prewarm()
}
```

Kotlin versions: compiler-plugin APIs are not stable across Kotlin releases. ghost-compiler-plugin is compiled against Kotlin 2.2.21 and verified in CI on 2.2.21, 2.3.21 and 2.4.0 (compat-tests/kotlin-consumer, -PkotlinVersion); GhostCompilerCompat bridges the APIs whose JVM signatures changed in 2.4. On any other Kotlin version the Gradle plugin still applies it but logs a warning. If a Native/Wasm compilation fails because of it, set `autoRegistration` to false and use `addRegistry`.

Rejected alternative: a generated top-level property with `@EagerInitialization` calling `addRegistry`. That annotation is deprecated, behaves differently per backend and gives no ordering guarantee across modules.

addRegistry flow

When you call `addRegistry`, Ghost walks `registry.getAllSerializers()` and preloads `serializerCache` and `serializerByName`. Call it once at startup, not per request.

8. Ghost.kt — Public runtime API

Central file:
ghost-serialization/src/commonMain/kotlin/com/ghost/serialization/Ghost.kt

Full public API table

Method	Description
<code>---</code>	<code>---</code>
<code>deserialize<T>(json: String)</code>	Decode UTF-8 → flat reader → serializer
<code>deserialize<T>(bytes: ByteArray)</code>	No intermediate String copy
<code>deserialize<T>(source: BufferedSource)</code>	Okio streaming
<code>deserialize(reader) / deserialize(flatReader)</code>	Direct use in generated code
<code>encodeToString<T>(value)</code>	String writer → String
<code>encodeToBytes<T>(value)</code>	Flat writer → ByteArray
<code>encodeAndDiscard<T>(value)</code>	Warm-up without result alloc
<code>serialize(sink, value)</code>	Bulk drain to Okio
<code>decodeFromBytes(bytes, KClass)</code>	Spring / Retrofit
<code>decodeFromSource(source, KClass)</code>	Streaming + KClass
<code>encodeToSink(sink, value, KClass)</code>	Encode with dynamic class
<code>getSerializer(KClass) / getSerializer(KType)</code>	Resolution + cache

addRegistry(registry)	Manual registration; optional on iOS/Wasm with the Gradle plugin (see §7)
prewarm()	Loads registries + warmUp serializers
getSerializerByName / getSerializerNames	Compiler bridge
throwError(msg)	Uniform IllegalArgumentException
Constants NOT_FOUND, MISSING_ANN	Error messages

Resolution order in getSerializerFromRegistries:

1. mutableRegistries (manual addRegistry)
2. discoverRegistries() — ServiceLoader JVM / empty on iOS and Wasm
3. registry.getSerializer(clazz) — generated sharded when
4. findLinkedSerializer(clazz) — @GhostSerializerLink associated object (Native/Wasm only; null on JVM)

Resolving serializer

Order:

1. serializerCache[KClass] (primitives + prewarm)
2. mutableRegistries (manual addRegistry)
3. discoverRegistries() (ServiceLoader JVM)
4. registry.getSerializer(clazz) → generated when
5. Compiler-plugin link via findAssociatedObject (Native/Wasm)

Deserialize — when to use each overload

```
// Most common: HTTP response already as String
val user = Ghost.deserialize<User>(responseBody)

// Better if you already have bytes (Retrofit/OkHttp without intermediate String)
val user = Ghost.deserialize<User>(bytes)

// Large streaming from Okio
val user = Ghost.deserialize<User>(bufferedSource)

// Advanced reader options (coercion, etc.) — requires internal OptIn
Ghost.deserialize<User>(json) { reader ->
    reader.coerceBooleans = true
}
```

Inside: resolveSerializer<User>() → ghostInternalUseFlatReader takes reader from pool → UserSerializer.deserialize(flatReader) → returns object.

Encode — when to use each overload

```
val s = Ghost.encodeToString(order)    // UI, logs, SharedPreferences
val b = Ghost.encodeToBytes(order)     // send over network without String
Ghost.serialize(okioSink, order)       // write directly to socket/file
Ghost.encodeAndDiscard(order)          // only warm JIT, discard bytes
```

`serialize(value)` without sink is an alias of `encodeToString(value)` for compatibility.

getSerializer — framework integration

Spring and Retrofit do not always have reified T. They use:

```
Ghost.getSerializer(Order::class)
Ghost.decodeFromBytes(bytes, Order::class)
```

For `List<Order>` / `Set<Order>` / `Map<String, Order>` in Retrofit (and Spring MVC/WebFlux), factories resolve `ParameterizedType` / `ResolvableType` and build `ListSerializer` / `SetSerializer` / `MapSerializer` around the element serializer. Ktor uses `Ghost.getSerializer(KType)` the same way.

Prewarm — why and when

```
// Application.onCreate (Android) or @PostConstruct (Spring)
Ghost.addRegistry(GhostModuleRegistry_my_app.INSTANCE) // iOS/Wasm: only so prewarm() can
walk the module
Ghost.prewarm()
```

`prewarm()` does three things: discovers registries (JVM), fills `serializerCache` with all module serializers, and calls `warmUp()` on each (dummy deserialize for JIT). The first real request pays less JIT compilation latency.

9. GhostSerializer — Dual contract

`GhostSerializer<T>` (contract/GhostSerializer.kt) is a fully abstract contract. Generated serializers extend `AbstractGhostSerializer<T>`, which supplies the defaults (Okio sink/source bridges, string-channel fallbacks, no-op `warmUp`). Hand-written serializers should extend it too, and override only what they need.

Why “dual” (two writers and two readers)

Ghost optimizes two distinct scenarios:

1. In-memory / HTTP body as bytes → `GhostJsonWriter` over a contiguous `FlatByteArrayWriter`, plus `GhostJsonFlatReader` on a `ByteArray`. Maximum throughput.
2. Okio streaming (files, chunks) → the same `GhostJsonWriter` over an Okio sink, plus `GhostJsonReader` on `BufferedSource`.

One writer class serves both output channels through `GhostByteSink`; it calls the flat buffer directly when that is the backend, so the in-memory path stays monomorphic. The two readers stay separate on purpose: unifying them measurably cost throughput on array-heavy payloads.

Methods each FooSerializer implements

Method	Main use
serialize(GhostJsonWriter, T)	encodeToBytes, serialize(sink), Retrofit request
serialize(GhostJsonStringWriter, T)	encodeToString (text channel)
deserialize(GhostJsonFlatReader)	Ghost.deserialize(bytes)
deserialize(GhostJsonReader)	streaming + options
warmUp()	called from prewarm()
typeName	readable key ("BenchUser")
isResilient	lists that skip bad elements

Mental example

```
Ghost.encodeToString(user)
    → BenchUserSerializer.serialize(flatWriter, user)
```

```
Retrofit response
    → BenchUserSerializer.deserialize(flatReader)
```

Always the same BenchUserSerializer object; different paths depending on buffer type.

PART IV — Internal runtime

End-to-end deserialize and encode pipeline

Deserialize (hot path)

```
JSON bytes
    → ghostInternalUseFlatReader (ThreadLocal pool)
    → GhostJsonFlatReader.reset(data)
    → FooSerializer.deserialize(reader)
        → beginObject()
        → loop: selectNameAndConsume(OPTIONS) // 0(1) trie
        → nextInt / nextString / skipValue
        → bitmask REQUIRED_MASK
        → primary constructor (multi-branch if applicable)
    → T
```

Encode (hot path)

```
value T
    → Ghost.resolveSerializer<T>()
    → ghostInternalEncodeWithWriter (WriterSinkPair pool)
    → FooSerializer.serialize(flatWriter, T)
        → pre-encoded writeNameHeader
        → writeInt / writeQuotedString
        → endObject()
    → FlatByteArrayWriter.toByteArray()
    → reset() — trims buffer if > maxWarmWriteBufferCapacity
```

Why two writers and two readers

Type	When	Cost
GhostJsonFlatReader	Ghost.deserialize, Retrofit, Spring body	Minimum: contiguous array
GhostJsonReader	Okio stream, coerceBooleans options	More flexible
GhostJsonWriter	encodeToBytes / serialize(sink)	Flat buffer (monomorphic hot path) or Okio sink
GhostJsonStringWriter	encodeToString	Contiguous CharArray path

10. GhostJsonFlatReader — Fast parser

GhostJsonFlatReader is the parser used in almost the entire hot path of Ghost. It reads a ByteArray start to end with a position index, without allocating objects per token.

Lifecycle in a typical deserialize

1. `reset(bytes)` — points at the response buffer (may be the same array from the pool).
2. `beginObject()` — expects `{`, prepares object context.
3. Loop `hasNext() / selectNameAndConsume(OPTIONS)` — identifies current field.
4. `nextInt()`, `nextString()`, etc. — consume value and advance position.
5. `skipValue()` — if JSON has fields your model does not (forward compatible).
6. `endObject()` — validates `}`.
7. Serializer checks mask and builds the data class.

selectNameAndConsume explained without jargon

Instead of comparing strings each iteration (if `(name == "email")`), the compiler precomputes `JsonReaderOptions` with a mini-hash of field names. At runtime:

- Returns 0 if the field is "id"
- Returns 1 if it is "name"
- Returns -1 if the object ended
- Returns -2 if unknown field → typically `skipValue()`

That is amortized $O(1)$ and JIT-friendly (fewer branches with strings).

Security limits

Parameter	What it protects	Typical value
<code>---</code>	<code>---</code>	<code>---</code>
maxDepth	Infinite nested JSON [][[]]	255
maxCollectionSize	Huge lists/maps (DoS)	50k Android, 1M JVM

If exceeded, clear exception. Does not limit total HTTP body size —

configure that in OkHttp/Spring.

Typical generated code

```
reader.beginObject()
while (true) {
    when (reader.selectNameAndConsume(OPTIONS)) {
        0 -> id = reader.nextInt()
        1 -> name = reader.nextString()
        -1 -> break
        -2 -> reader.skipValue()
    }
}
reader.endObject()
```

11. GhostJsonReader — Streaming parser

parser/GhostJsonReader.kt — on Okio / streaming.

- String pool for repeated names
 - GhostDiscriminatorPeeker for standard sealed
 - Used when API requests GhostJsonReader in options lambda
- Pools: ThreadLocal JVM/Android, @ThreadLocal iOS in Ghost.*.kt.

12. Writers — FlatByteArrayWriter

writer/bytes/FlatByteArrayWriter.kt + writer/bytes/GhostJsonWriter.kt

- Growing flat buffer
- reset() after each encode — releases capacity above maxWarmWriteBufferCapacity
- JVM: 8 MB warm cap; Android/Native/Wasm: 4 MB

// After encode, reset keeps buffer up to cap

```
fun reset() {
    // needsComma=false, depth=0
    // if capacity > maxWarm → shrink to 8KB initial
}
```

Pre-encoded headers: bulk write of "field": without UTF-8 encode per field on hot path.

GhostJsonWriter — Okio variant for serialize(sink).

13. GhostPools and scratch buffers

GhostPools.kt — expect/actual

- acquireScratchBuffer(size) / releaseScratchBuffer
- Size tiers to avoid alloc in number→string conversion and ASCII escape

Instance pools (in `Ghost.jvm.kt` / `.android.kt` / `.ios.kt`):

- `GhostJsonFlatReader + ByteArrayGhostSource`
- `GhostJsonReader`
- `WriterSinkPair` (flat writer + sink)

`runSynchronized` on registry caches.

14. GhostHeuristics — Platform limits

`GhostHeuristics` (expect/actual per target) defines compiled limits per platform. They are not flags the user changes in `Ghost.deserialize()` today.

Row-by-row table

Property	Android	iOS/Native	JVM server	Practical meaning
---	---	---	---	---
<code>maxCollectionSize</code>	50,000	500,000	1,000,000	Max elements when parsing List or Map
<code>maxWarmWriteBu</code>	4 MB	4 MB	8 MB	After encode, if internal buffer exceeds this, shrinks to ~8 KB
<code>maxDepth</code>	255	255	255	Max nesting of { [

`maxCollectionSize`: protects against `{"items":[...millions...]}`. On JVM server the limit is higher because legitimate batch APIs can be large; on mobile it is stricter for RAM.

`maxWarmWriteBufferCapacity`: if you serialize a 6 MB JSON once, the buffer grows; after `reset()`, Ghost does not keep 6 MB forever on mobile (4 MB cap) but on JVM may keep up to 8 MB to avoid realloc on the next similar response. This improved server write benchmarks without inflating Android RAM.

HTTP body: a 50 MB POST can enter memory whole if `OkHttp/Spring` allow it; Ghost will parse until `maxCollectionSize` or system memory says stop. Configure HTTP limits outside Ghost.

Collections supported at runtime (verified in code)

Type	Support
---	---
<code>List<T></code>	Yes — <code>ListSerializer</code> via <code>getSerializer(KType)</code>
<code>Map<String, V></code>	Yes — <code>MapSerializer</code>
<code>Set<T></code>	Yes — <code>SetSerializer</code> via <code>getSerializer(KType)</code> ; HTTP adapters unwrap top-level <code>Set<T></code> (Retrofit / Spring / Ktor)

15. Primitive and collection serializers

serializers/PrimitiveSerializers.kt, ListSerializer, MapSerializer, etc.

Ghost resolves `Int`, `String`, `List<T>`, `Set<T>`, `Map<String,V>` with built-in serializers before the registry.

For generics: `getSerializer(KType)` builds or caches `ListSerializer` / `SetSerializer` / `MapSerializer` around the element serializer.

16. Sealed, inferred, multi-branch

Sealed with discriminator (flow to visualize)

JSON arrives → sealed serializer reads type field (or what you define) → when picks sub-serializer → `ClickSerializer.deserialize` or `ScrollSerializer.deserialize`.

Advantage: explicit, easy to debug. Disadvantage: backend must always send the discriminator.

Inferred (no type field)

The compiler analyzes which field combination is unique per subclass. Generates a tree of if/when like: “if x and y present → Click; if only delta → Scroll”. Single JSON pass.

Useful when migrating legacy APIs. More generated bytecode and more branches in the serializer.

Multi-branch constructors (Kotlin defaults)

`BenchUser` has `isActive = true`, `role = VIEWER`, `bio = null`. They may be missing in JSON. Instead of:

```
// What Ghost does NOT want on hot path
BenchUser(id, name, email, score).copy(isActive = true, role = VIEWER)
```

The compiler generates up to 2^N branches if `((mask and X) == X)` return `BenchUser(...)` (one constructor call per branch) instead of chaining `.copy()`. Fewer temporary objects, less GC.

PART V — HTTP adapters

17. ghost-gradle-plugin

`GhostPlugin.kt` when applied:

1. If KSP → adds ghost-compiler to `ksp` / `kspCommonMainMetadata`
2. If KMP → ghost-serialization + ghost-api on `commonMain`
3. If Android/JVM → runtime implementation + api
4. afterEvaluate: if Retrofit/Ktor on classpath → ghost-retrofit / ghost-ktor

\$7

```
ghost {
    version.set("1.3.3")
    autoInjectKtor.set(true)
    autoInjectRetrofit.set(true)
    autoRegistration.set(true)
}
```

Plugin id: com.ghostserializer.ghost. DEFAULT_VERSION in plugin = 1.3.3.

18. ghost-retrofit

Full configuration

```
dependencies {
    implementation("com.ghostserializer:ghost-retrofit:1.3.3")
}

interface ApiService {
    @GET("user/{id}")
    suspend fun getUser(@Path("id") id: Int): User // User must have @GhostSerialization

    @POST("users")
    suspend fun createUser(@Body user: User): User
}

val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GhostConverterFactory.create()) // before Gson if both exist
    .client(okHttpClient)
    .build()
```

What the factory does internally

Response: reads `ResponseBody` into a reusable buffer (scratch), then `ghostInternalUseFlatReader` → `serializer.deserialize(flatReader)`. Does not go through intermediate UTF-16 String if the body is already bytes.

Request: `ghostInternalEncodeWithWriter { w -> serializer.serialize(w, value) }` → `RequestBody` with `application/json`.

Cache: `ConcurrentHashMap<Type, GhostSerializer>` — first call per type resolves; later calls are $O(1)$.

null body: if the Retrofit method allows nullable body and null arrives, converter returns null without serializing.

Coexisting with Gson

If you add `GhostConverterFactory` before `GsonConverterFactory`, Retrofit tries Ghost first. Only types with a Ghost serializer use Ghost; the rest fall

through to Gson.

19. ghost-ktor

GhostContentConverter.kt + GhostKtorExtensions.kt

```
val client = HttpClient {
    install(ContentNegotiation) {
        ghost() // Ktor 3.3.x+
    }
}
```

Same pool + flat reader/writer pattern. Ktor 3 in consumer apps may need a custom converter (see android test app GhostKtor3Converter).

20. ghost-spring-boot-starter

Dependencies

```
plugins {
    id("com.ghostserializer.ghost") version "1.3.3"
}
dependencies {
    implementation("com.ghostserializer:ghost-spring-boot-starter:1.3.3")
}
```

The ghost plugin in the same module as your @RestController and DTOs with @GhostSerialization.

Auto-configuration (Boot 3.4+)

Three open classes imported via META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports:

Class	Effect
----	----
GhostAutoConfiguration	Marker / imports
GhostWebMvcAutoConfiguration	Registers GhostHttpMessageConverter at start of list
GhostWebFluxAutoConfiguration	Reactive encoder/decoder codecs for WebFlux

No ghost.* in application.properties. Everything is automatic if the DTO has a serializer.

How it interacts with Jackson

Spring Boot brings Jackson by default. Ghost inserts its converter at index 0:

- If Ghost.getSerializer resolves the declared type (including top-level List/Set/Map of registered DTOs) → Ghost serializes/deserializes.
- If no Ghost serializer → Jackson continues with the DTO as before.

You can have in the same project: JPA entities with Jackson and API DTOs with Ghost. Collection endpoints such as `List<OrderDto>` stay on Ghost when `OrderDto` is registered.

Typical controller (no special changes)

```
@RestController
@RequestMapping("/api")
class OrderController(private val service: OrderService) {

    @GetMapping("/orders/{id}")
    fun get(@PathVariable id: Long): OrderDto = service.find(id)

    @PostMapping("/orders")
    fun create(@RequestBody dto: OrderDto): OrderDto = service.create(dto)
}

@GhostSerialization
data class OrderDto(val id: Long, val items: List<LineItem>)
```

Body size limit (important)

Ghost does not cut 100 MB bodies. Configure in Spring:

```
spring:
  codec:
    max-in-memory-size: 10MB
```

And/or limits in nginx/API gateway. See PART VII security model.
Spring Boot 3.4.5 verified in starter tests.

21. ProGuard / R8

Ghost ships R8 keep rules in three layers — you usually do not hand-copy rules into `proguard-rules.pro`:

1. Consumer rules (automatic) — merged from the `ghost-api` and `ghost-serialization` AARs when `minifyEnabled` true. They keep `@GhostSerialization`, `@GhostProtoSerialization`, `@GhostYamlSerialization` DTOs, `com.ghost.serialization.generated.**`, and all `GhostSerializer` / `GhostYamlSerializer` companions.

2. KSP-generated rules (per module) — `build/generated/ksp/.../META-INF/proguard/ghost-serialization.pro` keeps that module's `GhostModuleRegistry_*` and its serializers.

3. App-specific rules (rare) — only if you use custom registries or non-Ghost reflection; see `[ghost-android-test-app](https://github.com/juanchurtado1991/ghost-and-roid-test-app)` for a release build with R8 enabled.

Release checklist: build a release variant and confirm

META-INF/proguard/ghost-serialization.pro is present under the module that runs KSP. If Ghost.prewarm() or Retrofit works in debug but fails in release with NOT_FOUND, R8 stripped something — re-sync Gradle and verify the Ghost plugin runs on every module that declares @GhostSerialization models.

22. Maven Central publishing

Publishable (publish.gradle.kts): ghost-* except benchmark, integration-test, playground.

From Mac for full KMP (iosArm64 + iosSimulatorArm64 + JVM + Android AAR).

```
./gradlew publishToMavenCentral
```

Uploads a deployment bundle to Central Portal in USER_MANAGED mode (staged, not released). Review and publish it manually at [central.sonatype.com](https://central.sonatype.com/publishing/deployments). To publish and release automatically instead, run ./gradlew publishAndReleaseToMavenCentral.

Coordinates: com.ghostserializer:*:1.3.3

From Linux: iOS variants may be missing on Central.

PART VI — CI, tests, and commands

23. CI and tests

Single workflow: .github/workflows/ci.yml

Job	Runner	Command	What it validates
---	---	---	---
test-jvm	ubuntu-latest	./gradlew ciTestJvm	Compiler, integration-test, retrofit, ktor, spring, plugin
test-android	ubuntu-latest	:ghost-serialization:te	Android runtime
test-ios	macos-26	kspCommonMainKotlin + iosSimulatorArm64Test	K/N iOS; fails if SKIPPED
consumer-compatible	macos-26	compatible-tests/kotlin-co against mavenLocal	Kotlin 2.2.21 consumer, Gradle plugin, zero-registration round trips on JVM/Android/iOS/Wasm
test-wasm	ubuntu-latest	wasmJsBrowserTest (api/serialization/ktor)	Wasm runtime

publish-check	ubuntu-latest	publishToMavenLocal	Maven packaging
		-PskipTests	

iOS CI: Kotlin/Native does not print "N tests completed". The workflow trusts Gradle exit code and detects SKIPPED/FAILED in the log.

ciTestJvm in root build.gradle.kts:

- ghost-api, ghost-serialization jvmTest
- ghost-compiler, ghost-compiler-plugin test
- ghost-integration-test
- ghost-retrofit, ghost-ktor jvmTest
- ghost-spring-boot-starter test
- ghost-gradle-plugin, ghost-playground test

Verified on this repo (Gradle test XML, October 2026): ciTestJvm runs 11,310 tests (8,537 of them in ghost-serialization:jvmTest, mostly fuzz-corpus replays and the YAML conformance suite); testAndroidHostTest adds 977; the Wasm suite runs 992. iOS runs the same common suite on the simulator in CI.

ghost-benchmark:run depends on ciTest (except -PskipTests).

Reference Gradle commands

Goal	Command
---	---
Local CI (JVM)	./gradlew ciTestJvm
Full CI (+ Android)	./gradlew ciTest
iOS (Mac only)	./gradlew :ghost-serialization:iosSimulatorArm64Test
View generated KSP code	./gradlew :ghost-integration-test:compileKotlin
Benchmark	./gradlew :ghost-benchmark:run
Benchmark without tests	./gradlew :ghost-benchmark:run -PskipTests
Playground (local wasm)	./gradlew :ghost-playground:wasmJsBrowserDevelopmentRun
Publish local	./gradlew publishToMavenLocal -PskipTests
Maven Central (staging, manual review)	./gradlew publishToMavenCentral
Maven Central (publish + auto-release)	./gradlew publishAndReleaseToMavenCentral
Regenerate manual PDF	.venv-pdf/bin/python scripts/build_ghost_manual_pdf.py

Toolchain: JDK 17, Kotlin/KSP per gradle/libs.versions.toml.

24. Test apps (your 3 repos)

App	What it validates
ghost-android-test-app	Gson/Moshi/KSer vs Ghost, Retrofit, Ktor3
ghost-spring-boot-test-app	Jackson vs Ghost WebFlux, benchmark.py
ghost-ios-test-app	XCFramework + GhostBridge + Codable

All use 1.3.3 Maven Central (no mavenLocal in final config).

25. End-to-end flow on Android

Step by step (from scratch)

1. settings.gradle.kts — pluginManagement { gradlePluginPortal() }
2. app/build.gradle.kts — id("com.ghostserializer.ghost") version "1.3.3"
3. Create data class with @GhostSerialization in the network package
4. Build → Make Project — verify UserSerializer.kt exists in app/build/generated/ksp/
5. Application.onCreate: Ghost.prewarm() (optional but recommended for high-traffic apps)
6. Retrofit: .addConverterFactory(GhostConverterFactory.create())

§21

What happens on the first Retrofit request

OkHttp receives bytes

- GhostConverterFactory
- ghostInternalUseFlatReader
- UserSerializer.deserialize
- User object in memory

No reflection on that path.

26. End-to-end flow on iOS

1. KMP shared module with ghost plugin + export serialization/api
2. assembleXCFramework on Mac
3. Swift imports framework
4. No registration code: the Gradle plugin links serializers at compile time (call Ghost.addRegistry(INSTANCE) only with autoRegistration = false or to use prewarm())
5. deserialize via bridge — prefer UTF-8 String vs raw bytes

27. End-to-end Spring flow

1. implementation ghost-spring-boot-starter
2. ghost plugin + ksp in same module as controllers
3. Annotated DTOs
4. Normal controllers @RequestBody / @ResponseBody
5. Ghost converter only for types with serializer

PART VII — Publishing, security, errors

Security model and limits

Ghost separates three policy layers:

1. Parser (Ghost): `maxCollectionSize`, `maxDepth` — anti-DoS on lists/maps and nesting.
2. Encode memory: `maxWarmWriteBufferCapacity` — does not retain huge buffers between requests.
3. HTTP (your app): body size, timeouts, reverse proxy.

A 100 MB HTTP body is not rejected automatically by Ghost; configure `OkHttp`, `spring.codec.max-in-memory-size`, `nginx`.

`GhostJsonException` includes approximate position in the buffer to debug malformed JSON or missing required fields.

28. Common errors

NOT_FOUND or “No serializer found”

Typical message: includes `NOT_FOUND` and the type’s `KClass`.

Causes and fix:

1. Class lacks `@GhostSerialization` → add annotation and recompile.
2. KSP did not run (broken clean, module without ghost plugin) → `./gradlew :app:kspDebugKotlin` or `rebuild`.
3. Model in another module without KSP in that module → each module with models needs plugin + `ksp`.
4. iOS/Wasm: the module was built without the Ghost Gradle plugin (or with `autoRegistration = false`) and `Ghost.addRegistry(GhostModuleRegistry_xxx.INSTANCE)` was not called. On these targets the error message itself says so.

§21

GhostJsonException — missing required field

Message: Required field 'email' missing in JSON

Means: JSON did not include a field that has no Kotlin default. Not a Ghost bug: API contract mismatch.

What to do: fix backend, make field nullable, or give Kotlin default.

GhostJsonException — malformed JSON

Position in the message points to approximate byte of error (missing comma, unclosed string). Useful to log raw request body.

maxCollectionSize exceeded

List or map in JSON exceeded platform limit (50k on Android). May be legitimate large JSON or DoS attack.

What to do: paginate API, or if you control the reader in custom decoder, adjust `maxCollectionSize` on that path (not on global pooled `Ghost.deserialize`).

Spring still uses Jackson for my DTO

`GhostHttpMessageConverter` only handles classes where `Ghost.getSerializer(clazz) != null`. If your `@RequestBody` Data lacks `@GhostSerialization` in the same module compiled with KSP, Jackson wins.

Fix: annotate DTO, apply ghost plugin to controller module, verify Ghost converter is registered (starter does index 0).

Plugin com.ghostserializer.ghost not found

Gradle does not resolve the plugin. Check `pluginManagement` in `settings.gradle.kts` with `gradlePluginPortal()`, version 1.3.3 on Maven Central, and sync again.

Compiler plugin warning: generated serializer not found

Message: Ghost: generated serializer fixtures.UserSerializer not found for fixtures.User; automatic Kotlin/Native and Kotlin/Wasm registration is skipped for this class.

Means: the compiler plugin ran, but KSP output for that class is not part of the same compilation (for example KSP only runs on another source set). The class still works through `Ghost.addRegistry`.

Fix: let the Ghost Gradle plugin wire KSP for every target, or register the module manually.

Native/Wasm build fails inside the Ghost compiler plugin

Usually a Kotlin version the plugin was not built for (the build logs a Ghost: the compiler plugin is tested with Kotlin ... warning). Set `ghost { autoRegistration.set(false) }` and call `Ghost.addRegistry` until a Ghost release supports that Kotlin version.

iOS: works in debug, fails in release

Almost always R8/ProGuard, or automatic registration disabled without an `addRegistry` call. Check generated rules in `META-INF/proguard/` of the module with KSP.

29. Quick API — cheat sheet

```
Ghost.deserialize<T>(json: String)
```

```
Ghost.deserialize<T>(bytes: ByteArray)
Ghost.encodeToString(value)
Ghost.encodeToBytes(value)
Ghost.serialize(sink, value) // Okio bulk write
Ghost.addRegistry(registry) // optional; iOS/Wasm link automatically via the Gradle plugin
Ghost.prewarm()
Ghost.getSerializer(MyClass::class)
```

30. Key files to read in the repo

Topic	Path
---	---
API	ghost-api/src/commonMain/kotlin/com/ghost/serialization
Processor	ghost-compiler/src/main/kotlin/com/ghost/serialization
Generator	ghost-compiler/src/main/kotlin/com/ghost/serialization
Runtime	ghost-serialization/src/commonMain/kotlin/com/ghost
Flat reader	ghost-serialization/src/commonMain/kotlin/com/ghost
Flat writer	ghost-serialization/src/commonMain/kotlin/com/ghost
JVM pools	ghost-serialization/src/jvmMain/kotlin/com/ghost/seria
iOS	ghost-serialization/src/iosMain/kotlin/com/ghost/serial
Retrofit	ghost-retrofit/src/main/kotlin/com/ghost/serialization/
Ktor	ghost-ktor/src/commonMain/kotlin/com/ghost/serializa
Spring	ghost-spring-boot-starter/src/main/kotlin/com/ghost/s
Plugin	ghost-gradle-plugin/src/main/kotlin/com/ghost/gradle/

31. Version 1.3.3 — relevant changes {#cap-31--version-1.3.1-cambios-relevantes}

- 1.3.3 is a consumer-compatibility fix release (see CHANGELOG.md):
- Defaults that reference a nested class compile again: a constructor default such as Lifecycle.Type.UNKNOWN, where Lifecycle is nested in the model, is now emitted fully qualified in the generated serializer (it failed with Unresolved reference since 1.3.0 on models with more than four defaults).
 - Android AARs no longer force compileSdk 36 on consumers: minCompileSdk in the AAR metadata is 21 (Ghost's minSdk) instead of following Ghost's own compileSdk.

1.3.2 highlights

- Kotlin 2.2.21 minimum (down from 2.4.0): KSP 2.3.x (2.3.12), Ktor 3.3.x+ (3.3.3). Verified in CI by compat-tests/kotlin-consumer, a standalone Kotlin 2.2.21 project consuming the published artifacts.
- §7
- Errors: JSON/YAML decode errors carry a JSONPath and a fix hint;

not-found errors on Native/Wasm explain registration.

- Performance (no allocation increase): lazy JSONPath reconstruction on error, monomorphic flat-sink writes, inlined buffer capacity checks, single-space token peek, whole-value string-pool hashing, 8-digit SWAR number parsing, primitive-array decode fast path.
- Correctness: Double/Float parsing and formatting are round-trip safe.
- Structure: one GhostJsonWriter/GhostYamlWriter per format over GhostByteSink; fully abstract GhostSerializer/GhostRegistry contracts with AbstractGhostSerializer/AbstractGhostRegistry providing the defaults.

1.3.1 highlights

- YAML: yaml-test-suite harness + fuzzing; reader compliance 96.06%; writer round-trip 100%.
- HTTP adapters: Spring/Retrofit top-level List/Set/Map unwrap (element scalars like List<String> included; Map keys must be String).
- Wasm / Safari: Ghost.encodeToString on JavaScriptCore uses UTF-8 flat writer + TextDecoder (char writer kept on Chrome/V8); playground string Speed Test $\sim 2.7\times$ KSER on Safari ([#16](https://github.com/juanchurtado1991/ghost-serializer/issues/16)).
- Hot path: shared reader/writer kernels; digit-walk / SetSerializer / WKT string-channel fixes; deserialize(bytes){options} $\rightarrow$ flat reader.
- Toolchain (unchanged from 1.3.0): Kotlin 2.4.0 / KSP 2.3.10 / Ktor 3.5.x; wasmJs on API/serialization/Ktor/playground.

1.3.0 baseline retained for context: unified JSON+YAML+Proto runtime, textChannel=true default, RFC 8259 encoding normalization, in-order prediction + SWAR on JVM/Native.

Earlier 1.2.x notes retained below for historical context:

- Collision Resolution for Overlapping Field Names (Issue #10): Fixed perfect hash table collisions for classes containing fields with identical sizes and overlapping prefixes/suffixes (e.g., modelCode vs modelName, dateCreated vs dateUpdated). Refactored compiler (PerfectHashFinder.kt) and runtime (JsonReaderOptions.kt) to utilize a zero-allocation while loop with polynomial accumulation (multiplier 31) over all remaining bytes of the field name when a collision is detected. This guarantees correct field dispatch and prevents compile-time KSP failures or runtime parsing/deserialization exceptions.

31.1. Version 1.2.3 — changes
{#cap-31-1--version-1.2.3-cambios}

- **KSP2 AST Scan & Compiler Optimizations:** Integrated full KSP2 compatibility (aligned with Kotlin 2.4.0 / KSP 2.3.10) for faster incremental builds and improved compile-time AST inspection.
- **Perfect Hashing & O(1) Field Lookup:** Optimized key dispatch tables utilizing precomputations of 4-byte hash slots inside `JsonReaderOptions` to minimize collisions during field lookup.
- **Streaming Key-Match Optimization:** Specialized `verifyKeyMatch` loop inside `GhostJsonReader` to branch on `isStreaming` once at loop start, enabling direct `ByteArray` or `BufferedSource` reads, yielding a 2x throughput boost on binary payloads.
- **Zero-Allocation Coercion:** Handled string-to-boolean/number coercions natively at byte level, bypassing intermediary string allocations.
- **Wiki & Badges Revamp:** Updated documentation badges and flowcharts to rasterized .png format for clean out-of-the-box rendering inside Android Studio's Markdown preview.

31.5. Version 1.1.17 — legacy changes

- No `maxPayloadBytes` in core (HTTP limits in `OkHttp/Spring/nginx`)
 - JVM warm buffer 8 MB (fix inflated `WRITE` benchmark)
 - Spring split open `AutoConfiguration` (Boot 3.4)
 - Centralized `ciTestJvm`
 - `SpringBootTest` in starter
 - CI: `ciTestJvm` + Android host tests on Linux, iOS on macOS, Wasm in the browser — verified by Gradle test XML
-

32. Glossary

- **KSP:** Kotlin Symbol Processing — compile-time codegen
 - **Flat path:** reader/writer on contiguous `ByteArray`
 - **Registry:** `clazz` → generated serializer map
 - **Shard:** split registry when for JVM limit
 - **Trie options:** O(1) structure for JSON field names
 - **ServiceLoader:** JVM discovery of `GhostRegistry`
-

33. GhostSerializationProcessor — real code

File:

```
ghost-compiler/src/main/kotlin/com/ghost/serialization/compiler/GhostSerializationProcessor.kt
```

The processor is the heart of compile-time. On each KSP round:

```

override fun process(resolver: Resolver): List<KAnnotated> {
    val symbols = resolver.getSymbolsWithAnnotation(
        C.STR_ANNOTATION_SERIALIZATION
    )
    val validClasses = symbols.filterIsInstance<KSClassDeclaration>().toList()
    validClasses.forEach { processClass(it) }
    if (validClasses.isNotEmpty()) {
        generateModuleRegistry()
        generateProGuardRules()
        generateServiceFile()
    }
    return unableToProcess.toList()
}

```

Registry name: KSP option ghost.moduleName → GhostModuleRegistry_<name>. If code lives in src/test, _Test suffix is added to avoid colliding with production registry.

Per annotated class outputs:

- com.your.package.UserSerializer (object implementing GhostSerializer<User>)
- Entry in internal registry map

Global outputs (once per module):

- GhostModuleRegistry_my_app.kt with INSTANCE
- META-INF/services/com.ghost.serialization.contract.GhostRegistry
- META-INF/proguard/ghost-serialization.pro

34. JsonReaderOptions — O(1) trie in detail

parser/JsonReaderOptions.kt precomputes a 1024-slot dispatch table.

```

class JsonReaderOptions(
    internal val rawBytes: Array<ByteArray>,
    internal val shift: Int,
    internal val multiplier: Int
) {
    internal val dispatch = IntArray(1024) { -1 }
    // init: hash of up to 4 bytes of field name → index
}

// In generated serializer:
private val OPTIONS = JsonReaderOptions.of(
    0, 31, "id", "name", "email"
)

```

In the deserialize loop:

```

when (reader.selectNameAndConsume(OPTIONS)) {
    0 -> id = reader.nextInt()
}

```

```

1 -> name = reader.nextString()
-1 -> break      // end of object
-2 -> reader.skipValue() // unknown field
}

```

Why it matters: comparing strings on every field of a large JSON destroys performance. Ghost compares buffer bytes with names precomputed in `rawBytes`.

35. Ghost.jvm.kt — pools and discovery

ThreadLocal pools (same idea on Android):

```

private val flatReaderPool = ThreadLocal<GhostJsonFlatReader>()
private val writerPool = ThreadLocal<WriterSinkPair>()

```

```

actual fun ghostInternalEncodeWithWriter(
    block: (GhostJsonWriter) -> Unit
): ByteArray {
    val pair = acquireFlatWriterPair()
    block(pair.writer)
    val result = pair.byteWriter.toByteArray()
    pair.byteWriter.reset()
    return result
}

```

`discoverRegistries()` on JVM:

1.

`Class.forName("com.ghost.serialization.generated.GhostModuleRegistry_Default").getField("INSTANCE")` (JVM fast path)

2. `ServiceLoader.load(GhostRegistry::class.java)` reads META-INF/services

That is why on Android/JVM you do not need `addRegistry` if KSP generated the service file and R8 did not remove it.

36. Ghost.ios.kt — critical differences

```

actual fun discoverRegistries(): Iterable<GhostRegistry> = emptyList()

```

§7

```

// shared/src/iosMain or bridge
fun initGhost() {
    Ghost.addRegistry(GhostModuleRegistry_my_app.INSTANCE)
    Ghost.prewarm()
}

```

Pools with `@ThreadLocal` instead of `java.lang.ThreadLocal`.

Synchronization with `objc_sync_enter/exit` for shared caches.

Framework export: the module with models + KSP must export

ghost-serialization and ghost-api in XCFramework export(...).

37. GhostConverterFactory — Retrofit flow

```
class GhostConverterFactory private constructor() : Converter.Factory() {

    override fun responseBodyConverter(type, annotations, retrofit):
Converter<ResponseBody, *>? {
        val serializer = getSerializerWithCache(type) ?: return null
        return Converter { body ->
            body.use {
                // Read stream → scratch buffer → flat reader
                ghostInternalUseFlatReader(scratch, offset) { reader ->
                    serializer.deserialize(reader)
                }
            }
        }
    }

    override fun requestBodyConverter(type, annotations, retrofit): Converter<*,
RequestBody>? {
        val serializer = getSerializerWithCache(type) ?: return null
        return Converter { value ->
            ghostInternalEncodeWithWriter { w ->
                serializer.serialize(w, value)
            }.toRequestBody(MEDIA_TYPE)
        }
    }
}
```

- `getSerializerWithCache` supports `List<T>` and `Map<String,V>` via `ParameterizedType`
- If no serializer → null → Retrofit tries another converter
- Media type: `application/json`

38. Ktor integration — ghost()

`GhostContentConverter.kt` implements Ktor's `ContentConverter`.

```
fun ContentNegotiation.Config.ghost() {
    register(ContentType.Application.Json, GhostContentConverter())
}
```

Same flat reader/writer path as Retrofit. The Gradle plugin can inject `ghost-ktor` automatically if it detects Ktor on classpath (`autoInjectKtor`).

Ktor 3: some apps use a different API; the android test app includes `GhostKtor3Converter` as reference if `ghost()` does not compile.

39. Spring — three auto-configurations

Class	Activates
GhostAutoConfiguration	Marker / imports
GhostWebMvcAutoConfiguration	GhostHttpMessageConverter in MVC
GhostWebFluxAutoConfiguration	Reactive codecs

GhostHttpMessageConverter:

```

override fun supports(clazz: Class<*>) =
    Ghost.getSerializer(clazz.kotlin) != null

override fun readInternal(clazz: Class<*>, inputMessage: HttpInputMessage) =
    Ghost.decodeFromBytes(inputMessage.body.readBytes(), clazz.kotlin)

override fun writeInternal(t: Any, outputMessage: HttpOutputMessage) {
    val ser = Ghost.getSerializer(t::class as KClass<Any>)!!
    val bytes = ghostInternalEncodeWithWriter { w ->
        ser.serialize(w, t)
    }
    outputMessage.body.write(bytes)
}

```

The converter is inserted at index 0 to take priority over Jackson on Ghost types.

40. integration-test sample models

Package:

ghost-integration-test/src/main/kotlin/com/ghost/serialization/integration/model/

Types to study in the repo (sources, not generated):

- BenchUser — typical user benchmark
- ComplexObject — deep nesting
- SmartEvent — sealed inferred
- DeviceCommand — sealed with discriminator
- HugeModel — >40 fields → FragmentedEmitter
- DeepNestedModel — JSON depth

After ./gradlew :ghost-integration-test:compileKotlin, open:

ghost-integration-test/build/generated/ksp/main/kotlin/

There you will see real *Serializer.kt and GhostModuleRegistry_Default.kt.

41. FragmentedEmitter — huge models

When a data class exceeds ~40 properties, the compiler splits

serialize/deserialize into fragments to avoid the 64 KB JVM bytecode limit per method.

Symptom if it did not exist: compile error on generated serializer for massive models (legacy APIs, ERP DTOs).

42. Multi-branch constructors (≤ 4 properties with default (MAX_DEFAULT_BRANCH_COUNT = 4))

If a constructor has up to 4 parameters with default, KSP can generate when (mask) branches that call the primary constructor once with the correct argument combination, avoiding:

```
// Anti-pattern Ghost avoids on hot path
User(id).copy(name = n)
```

This reduces allocations on deserialize of objects with many optional fields.

43. Sealed inferred vs discriminator

With discriminator (discriminator = "type"):

```
{"type": "click", "x": 10, "y": 20}
```

Inferred (inferred = true): compiler generates decision tree from which fields appear:

```
{"x": 10, "y": 20}
```

No type field. Useful for inconsistent APIs; more generated code and more serializer branches.

44. Resilient and Fallback at runtime

@GhostResilient on enum: unknown JSON value → null or first value per generated policy.

@GhostFallback on sealed: default subclass if discriminator does not match.

GhostSerializer.isResilient: lists that tolerate malformed elements (skip instead of abort).

45. ghostInternalUseFlatReader — decode pipeline

```
inline fun <T> ghostInternalUseFlatReader(
    bytes: ByteArray,
    limit: Int = bytes.size,
    block: (GhostJsonFlatReader) -> T
): T {
    // Acquire reader from pool, reset(bytes), run block, return
```

```
}
```

`Ghost.deserialize<T>(json)` converts `String` → UTF-8 bytes → flat reader → `serializer.deserialize(reader)`.

`Ghost.deserialize<T>(bytes)` skips `String` conversion if you already have bytes (Retrofit, OkHttp).

46. `encodeToString` vs `serialize(sink)`

API	Output	Use
<code>encodeToString</code>	UTF-8 String	UI, logs, tests
<code>encodeToBytes</code>	ByteArray	network, binary cache
<code>serialize(sink)</code>	Okio BufferedSink	streaming to socket/file without full intermediate copy

`encodeToBytes` and `serialize(sink)` use `GhostJsonWriter` over a pooled `FlatByteArrayWriter` (the sink variant drains it in one bulk write); `encodeToString` uses `GhostJsonStringWriter`.

47. Gradle — dependencies the plugin injects

KMP commonMain:

- `com.ghostserializer:ghost-serialization`
- `com.ghostserializer:ghost-api (api)`

KSP:

- `com.ghostserializer:ghost-compiler` on `ksp` / `kspCommonMainMetadata` / `targets`

Kotlin compiler plugin (KMP, Native/Wasm compilations only):

- `com.ghostserializer:ghost-compiler-plugin`, unless `ghost { autoRegistration.set(false) }`

Conditional afterEvaluate:

- Retrofit on classpath → `ghost-retrofit`
- Ktor on classpath → `ghost-ktor`

```
ghost {
    version.set("1.3.3") // or omit if plugin brings DEFAULT_VERSION
}
```

48. R8 / ProGuard — generated rules

The processor writes rules that keep:

- Generated `*Serializer` classes
- `GhostModuleRegistry_*` and its `INSTANCE` field
- `GhostRegistry` interfaces for `ServiceLoader`

On Android, consumer AAR rules (automatic) plus KSP output keep:

```
-keep @com.ghost.serialization.annotations.GhostSerialization class * { *; }
-keep @com.ghost.serialization.annotations.GhostProtoSerialization class * { *; }
-keep @com.ghost.serialization.annotations.GhostYamlSerialization class * { *; }
-keep class com.ghost.serialization.generated.** { *; }
-keep class * implements com.ghost.serialization.contract.GhostSerializer { *; }
-keep class * implements com.ghost.serialization.yaml.contract.GhostYamlSerializer { *; }
```

Without this (or with KSP disabled on a module): `NOT_FOUND` in release even if debug works.

49. Benchmark and CI — useful commands

```
# Monorepo JVM tests (like CI)
./gradlew ciTestJvm

# Benchmark (requires prior tests except -PskipTests)
./gradlew :ghost-benchmark:run

# Regenerate serializers locally
./gradlew :ghost-integration-test:compileKotlin

# PDF of this manual
.venv-pdf/bin/python scripts/build_ghost_manual_pdf.py
```

PDF output: docs/Ghost-Serialization-Manual-1.3.3.pdf (A5 format).

50. Study checklist (40 min on phone)

1. Read ch. 1–4 (philosophy + annotations)
2. Read ch. 6–7 (generated serializer + registry)
3. Read ch. 9–12 (Ghost API + flat reader/writer)
4. Read ch. 35–36 (JVM vs iOS — critical)
5. Adapter for your stack: 37 Retrofit, 38 Ktor, 39 Spring
6. On PC open a generated `*Serializer.kt` and compare with ch. 6
7. Review ch. 28 common errors before production integration

PART VIII — integration-test catalog and generated code

Module `:ghost-integration-test` is the compiler and runtime regression lab. After `./gradlew :ghost-integration-test:compileKotlin`, inspect:

`ghost-integration-test/build/generated/ksp/main/kotlin/`

VIII.1 Full model catalog

96 types annotated with @GhostSerialization in src/main + src/test (includes sealed subclasses). 26 test classes in src/test/kotlin/com/ghost/serialization/integration/.

A. Benchmark and real API shape

Model	File	What it validates
----	----	----
BenchUser	BenchUser.kt	Multi-branch (3 defaults), nested enum
BenchResult	BenchResult.kt	
BenchmarkMetrics	BenchmarkMetrics.kt	Benchmark response
ComplexResponse	ComplexResponse.kt	Aggregated metrics
ApiUserEvent	ApiUserEvent.kt	Composite API payload
ApiProductConfig	ApiProductConfig.kt	User event
ExtremeMetadata	ExtremeMetadata.kt	Product config
Address	Address.kt	Bulky metadata
Category	Category.kt	Simple nesting
Tag	Tag.kt	Catalog
Priority	Priority.kt	Tags
UserRole	UserRole.kt	Priority enum
		Role enum (used by BenchUser)

Test: GhostStressTest, GhostLibraryMethodTest

B. Nesting, graphs, and generics

Model	What it validates
----	----
ComplexObject	Deep multi-level object
NestedContainer	Nested container
RecursiveNode	Controlled recursion
RecursiveGraphNode	Recursive graph
NestedGenericModel	Nested generics
DeepGenericModel	Deep generics
MapEdgeCaseModel	Map key/value edge cases

Test: GhostRobustnessTest, GhostAdvancedTypesTest, GhostTypeSystemTest

C. JVM fragmentation (>40 fields)

Model	Approx. fields	Emitter
----	----	----
Object40	40	StandardEmitter (limit)
Object41	41	FragmentedEmitter
GodObject	massive	FragmentedEmitter + FragmentedSerializeEmitter
WideModel	wide	Serialize fragmentation

Test: GhostBoundaryFragmentationTest, GhostGodObjectTest

D. Structural transformations

Model	Key annotation
<code>---</code>	<code>---</code>
FlattenedModel	@GhostFlatten
DeepFlattenedModel	deep flatten
WrappedModel	@GhostWrap
MixedStructuralModel	mix flatten/wrap
WrapSharedPathModel	shared paths
StructuralCollisionModel	path collisions
CollisionChild	collision child
CollisionModel	key collision

Test: `GhostStructuralTransformationTest`

E. Polymorphism with discriminator

Root / subclasses	Discriminator
<code>---</code>	<code>---</code>
SmartDevice → Light, Thermostat, UnknownDevice	type + @GhostFallback
ApiEventDefault → Login, Logout	implicit default
ApiEventExplicitType → Login, Logout	explicit type
GhostKindEvent → Created, Deleted, Updated	kind
StripeObject → Charge, Refund	object
JsonLdNode → Person, Organization	JSON-LD @type
GhostShape → Circle, Square	geometric sealed
GhostDiscriminatorTestPayload	test payload

Test: `GhostCustomDiscriminatorTest`, `GhostResilienceAndFallbackTest`

F. Inferred polymorphism (test sources only)

Model	Test
<code>---</code>	<code>---</code>
SmartEvent , TempEvent , HumidityEvent , MixedEvent , MotionEvent	GhostInferredPolymorphismTest
DeviceCommand → Reboot, SetBrightness, UpdateFirmware	GhostInferredPolymorphismTest
InferredNestedContainer	nested inferred
HugeModel , DeepNestedModel , MassiveInferredRoot	GhostProductionHardeningTest

Generated under: `build/generated/ksp/test/kotlin/`

G. Resilience, coercion, and enums

Model	Behavior
<code>---</code>	<code>---</code>
ResilientItem	@GhostResilient items
DeepResilientModel	deep resilient
SmartHome , HomeConfig , HomeStatus	IoT resilient
ResilientEnumModel	unknown enum
CoercionStressModel	strings→numbers

BooleanCoercionModel	0/1 → bool
GhostStandardsEnum, GhostEnumWrapper	enum standards
Test:	GhostCoercionTest, GhostEnumResilienceTest, GhostEnumStandardsTest

H. Custom coders and external types

Model	Mechanism
----	----
CustomCoderStressModel	@GhostEncoder/@GhostDecoder stress
CustomDateUser, LegacyUser	legacy dates
EncoderHexUtils, EncoderDateUtils, EncoderBooleanUtils	providers
ContextualModel, ModelWithExternal	3rd-party types
ExternalColor + ExternalColorSerializer	manual serializer
ExternalDate + ExternalDateSerializer	contextual date

Test: GhostCustomCoderTest, GhostContextualTest

I. Value classes and tokens

Model	Detail
----	----
UserId	@JvmInline value class
UserWithValueClass	transparent wrapper
GhostUserToken	typed token

Test: GhostValueClassTest

J. Nullability, defaults, evolution

Model	Detail
----	----
NullabilityStressModel	null on all types
DefaultValueNullModel	default + null
NullablePrimitives	nullable primitives
CollectionOfNulls	List with nulls
EvolutionModel	new/old fields
IgnoreModel	@GhostIgnore

Test: GhostNullabilityStressTest, GhostHardeningTest, GhostFinalHardeningTest

K. Naming, Unicode, reserved words

Model	Detail
----	----
NamingModel	@GhostName aliases
ReservedWordModel	reserved JSON names
UniCodeModel	complex UTF-8
EmojiKeyModel	emoji keys

Test: GhostFeatureTest, GhostEliteHardeningTest

L. Malice, DoS, and large strings

Model	Detail
---	---
MaliceModel	malicious payloads
LargeStringModel	huge strings
StressMetrics	stress metrics
DecimalStress	decimal precision

Test: GhostMaliceTest, GhostProductionHardeningTest

M. Concurrency and multi-branch

Topic	Detail
---	---
BenchUser	3 parameters with default → 8 constructor branches
GhostConcurrencyExpansionTest	pools + threads

Test: GhostMultiBranchConstructorTest, GhostConcurrencyExpansionTest

Test class → responsibility map

Test class	Focus
---	---
GhostStressTest	throughput / stability
GhostRobustnessTest	rare types
GhostBoundaryFragmentationTest	40/41 fields
GhostGodObjectTest	GodObject
GhostStructuralTransformationTest	flatten/wrap
GhostInferredPolymorphismTest	inferred
GhostCustomDiscriminatorTest	custom discriminator
GhostResilienceAndFallbackTest	resilient + fallback
GhostCoercionTest	type coercion
GhostCustomCoderTest	encoder/decoder
GhostContextualTest	external
GhostValueClassTest	inline classes
GhostNullabilityStressTest	nulls
GhostMaliceTest	parser security
GhostProductionHardeningTest	massive inferred
GhostMultiBranchConstructorTest	bitmask branches
GhostConcurrencyExpansionTest	threads
GhostLibraryMethodTest	API parity
GhostTypeSystemTest	Map/List edge
GhostAdvancedTypesTest	generics
GhostFeatureTest	naming
GhostEliteHardeningTest	unicode
GhostHardeningTest / GhostFinalHardeningTest	evolution
GhostEnumResilienceTest / GhostEnumStandardsTest	enums

VIII.2 Walkthrough: BenchUserSerializer line by line

Source model (KSP input)

```
@GhostSerialization
data class BenchUser(
    val id: Int,
    val name: String,
    val email: String,
    val score: Double,
    val isActive: Boolean = true,
    val role: UserRole = UserRole.VIEWER,
    val bio: String? = null
)
```

Required JSON fields: id, name, email, score (bits 1+2+4+8 = mask 15L).

Optional with Kotlin default: isActive, role, bio → compiler generates multi-branch (8 constructor combinations).

Generated output (location)

build/generated/ksp/main/kotlin/com/ghost/serialization/integration/mode
l/BenchUserSerializer.kt

Block 1 — Header and contract (L1-38)

```
@file:Suppress("UNUSED_VARIABLE", "UNCHECKED_CAST", "UNUSED_EXPRESSION", ...)
@file:OptIn(InternalGhostApi::class)
```

```
public object BenchUserSerializer : GhostSerializer<BenchUser> {
    override val typeName: String = "BenchUser"
```

- object (not class): zero alloc on lookup; singleton.
- typeName: key in GhostModuleRegistry and getSerializerByName.

Block 2 — JsonReaderOptions (L40-48)

```
private val OPTIONS = JsonReaderOptions.of(0, 31,
    "id", "name", "email", "score", "isActive", "role", "bio")
```

- 1024-slot trie; hash of first 4 bytes of field name.
- At runtime: selectNameAndConsume(OPTIONS) returns index 0..6, -1 end, -2 unknown.

Block 3 — Pre-encoded headers (L50-62)

```
private val H_ID: ByteString = "\"id\"".encodeUtf8()
private val H_EMAIL: ByteString = "\"email\"".encodeUtf8()
private val H_NAME: ByteString = "\"name\"".encodeUtf8()
// (and H_SCORE, H_ISACTIVE, H_ROLE, H_BIO the same way)
```

- Serialize writes "id": without re-encoding UTF-8 per request.
- writeField(H_ID, value.id) concatenates header + value.

Block 4 — Deserialize: variables and mask (L67-76)

```
var _id: Int = 0
var _name: String? = null
var _role: UserRole? = null
var _bio: String? = null
var _mask0 = 0L
reader.beginObject()
```

- One temporary variable per property.
- `_mask0` accumulates bits: id=1, name=2, email=4, score=8, isActive=16, role=32, bio=64.

Block 5 — Field loop (L77-114)

```
while (true) {
    val index = reader.selectNameAndConsume(OPTIONS)
    when (index) {
        0 -> { _id = reader.nextInt(); _mask0 = _mask0 or 1L }
        1 -> { _name = reader.nextString(); _mask0 = _mask0 or 2L }
        5 -> { _role = UserRoleSerializer.deserialize(reader); _mask0 = _mask0 or 32L }
        6 -> { _bio = if (reader.isNextNullValue()) { reader.consumeNull(); null }
                else reader.nextString(); _mask0 = _mask0 or 64L }
        -1 -> break
        -2 -> reader.skipValue()
    }
}
reader.endObject()
```

- Delegation: `UserRole` uses its own `UserRoleSerializer` (generated enum).
- Nullable: explicit `isNextNullValue` branch for `bio`.
- Unknown: `skipValue()` forward-compatible.

Block 6 — Required field validation (L116-126)

```
if ((_mask0 and 15L) != 15L) {
    if ((_mask0 and 1L) == 0L) reader.throwError("Required field 'id' missing in JSON")
    else if ((_mask0 and 2L) == 0L) reader.throwError("Required field 'name' missing in JSON")
    // (same for email and score)
}
```

- One AND on aggregated mask; specific message for first missing field.
- Throws `GhostJsonException` with buffer position.

Block 7 — Multi-branch constructors (L127-200)

```
if ((_mask0 and 112L) == 112L) {
    return BenchUser(id=_id, name=_name!!, email=_email!!, score=_score,
        isActive=_isActive, role=_role!!, bio=_bio)
}
if ((_mask0 and 48L) == 48L) {
```

```

    return BenchUser(id=_id, name=_name!!, email=_email!!, score=_score, isActive=_isActive,
role=_role!!)
}
// Other branches: 112L (all optional), 80L, 96L, 16L, 32L, 64L...
return BenchUser(id=_id, name=_name!!, email=_email!!, score=_score) // required only

```

- 112L = 16+32+64 → isActive + role + bio present.
- Each branch calls the primary constructor once with the exact combination of arguments present in JSON.
- Avoids chained .copy() on hot path (1.1.16+ optimization).

Block 8 — Flat deserialize (L206-340)

Duplicate logic for GhostJsonFlatReader — same structure, different reader (contiguous byte array). Ghost.deserialize(bytes) uses this path via pool.

Block 9 — Serialize (L342-370)

```

override fun serialize(writer: GhostJsonWriter, value: BenchUser) {
    writer.beginObject()
    writer.writeField(H_ID, value.id)
    writer.writeField(H_NAME, value.name)
    writer.writeField(H_EMAIL, value.email)
    writer.writeField(H_SCORE, value.score)
    writer.writeField(H_ISACTIVE, value.isActive)
    writer.name(H_ROLE)
    UserRoleSerializer.serialize(writer, value.role)
    if (value.bio != null) {
        writer.writeField(H_BIO, value.bio)
    }
    writer.endObject()
}

```

- Nullable optional fields: omitted if null (bio).
- Fields with Kotlin default but present: always serialized if not null.

Block 10 — warmUp() (L372-385)

```

override fun warmUp() {

    deserialize(GhostJsonReader("""{"id":0,"name":"","email":"","score":0.0}""").encodeToByteArray())

    deserialize(GhostJsonFlatReader("""{"id":0,"name":"","email":"","score":0.0}""").encodeToByteArray())
}

```

- Ghost.prewarm() invokes this to JIT/compile paths before real traffic.

Full flow in your app

```

Ghost.deserialize<BenchUser>(json)
→ resolveSerializer → BenchUserSerializer (registry)

```

```
→ ghostInternalUseFlatReader(bytes) { reader ->
    BenchUserSerializer.deserialize(reader) // blocks 5-7
}
```

VIII.3 integration-test generated registry

File:

build/generated/ksp/main/kotlin/com/ghost/serialization/generated/GhostModuleRegistry_Default.kt

```
// Conceptual structure (simplified)
class GhostModuleRegistry_Default : GhostRegistry {
    companion object { val INSTANCE = GhostModuleRegistry_Default() }

    override fun <T : Any> getSerializer(clazz: KClass<T>): GhostSerializer<T>? =
        when (clazz) {
            BenchUser::class -> BenchUserSerializer as GhostSerializer<T>
            Address::class -> AddressSerializer as GhostSerializer<T>
            BenchUser::class -> BenchUserSerializer as GhostSerializer<T>
            Address::class -> AddressSerializer as GhostSerializer<T>
            else -> null
        }

    override fun getAllSerializers(): Map<KClass<*>, GhostSerializer<*>> by lazy {
        // full map for prewarm()
    }
}
```

ServiceLoader:

META-INF/services/com.ghost.serialization.contract.GhostRegistry points to this class.

Test registry: models in src/test generate
 GhostModuleRegistry_Default_Test (_Test suffix when
 ghost.moduleName is Default and file is in test/).

Local inspection:

```
./gradlew :ghost-integration-test:compileKotlin
ls ghost-integration-test/build/generated/ksp/main/kotlin/com/ghost/serialization/
```

51. Maven artifacts table 1.3.3

```
com.ghostserializer:ghost-api
com.ghostserializer:ghost-serialization
com.ghostserializer:ghost-compiler (KSP)
com.ghostserializer:ghost-compiler-plugin (Kotlin compiler plugin, Native/Wasm)
com.ghostserializer:ghost-gradle-plugin
com.ghostserializer:ghost-retrofit
com.ghostserializer:ghost-ktor
```

com.ghostserializer:ghost-spring-boot-starter

Plugin id: com.ghostserializer.ghost version aligned with libraries.

Factual verification (aligned with code 1.3.3)

This manual was cross-checked against the ghost-serializer repository on the local working branch:

Manual claim	Source in repo
---	---
Registry sharding at 500 entries	GhostEmitterConstants.REGISTRY_CHUNK_SIZE = 500
Method fragmentation >40 fields	DEFAULT_CHUNK_SIZE = 40 in GhostEmitterConstants
Multi-branch up to 4 defaults	MAX_DEFAULT_BRANCH_COUNT = 4 in StandardEmitter.kt
maxCollectionSize Android 50k / JVM 1M / Nat 500k	GhostHeuristics.android.kt, .jvm.kt, .native.kt
maxWarmWriteBuffer 4 MB mobile / 8 MB JVM	same GhostHeuristics.*.kt files
maxDepth 255	GhostJsonNumericLimits.MAX_DEPTH
11,310 tests in ciTestJvm, 977 Android host, 9 Wasm	Gradle build/test-results XML, October 2026
Kotlin 2.2.21 minimum	gradle/libs.versions.toml kotlin-sdk = "2.2.21"; compat-tests/kotlin-consumer in CI
Spring Boot 3.4.5 in tests	gradle/libs.versions.toml spring-boot = "3.4.5"
Ktor 3.3.3	gradle/libs.versions.toml ktor = "3.3.3"
iOS/Wasm without ServiceLoader	Ghost.nonJvm.kt → discoverRegistries() = emptyList(); serializers resolve via findLinkedSerializer (GhostSerializerLinkLookup.native.kt / .wasmJs.kt)
JVM registry fast-path	Ghost.jvm.kt → Class.forName + ServiceLoader
Native/Wasm serializer link	ghost-compiler-plugin → GhostSerializerLinkExtension.kt
List/Map/Set at runtime	Ghost.kt → ListSerializer, MapSerializer, SetSerializer

If you upgrade the Ghost version, cross-check these files again before trusting exact numbers.

Appendix: API Reference

This section documents the public API of Ghost Serialization, derived from the

KDoc comments in the source code (version 1.3.3).

A.1 Ghost — Main entry point

object Ghost in package com.ghost.serialization.

Central entry point for Ghost Serialization. Manages modular registry discovery

and serialization/deserialization across all platforms.

Serialization

Method	Description
---	---
<code>serialize(sink, value)</code>	Encodes value and writes the resulting JSON payload into the given Okio BufferedSink. Encodes through GhostJsonWriter over a pooled flat buffer, then drains it in a single bulk write with no Okio segment overhead.
<code>serialize(value)</code>	Convenience alias for <code>encodeToString</code> .
<code>encodeToString(value)</code>	Serializes value to an in-memory JSON String. Writes through the pooled GhostJsonStringWriter (contiguous CharArray), avoiding Okio segments and an intermediate UTF-8 byte buffer.
<code>encodeToBytes(value)</code>	Serializes value to a UTF-8 JSON ByteArray. Skips intermediate string encoding/decoding.
<code>encodeAndDiscard(value)</code>	Serializes value discarding the output. Useful for JIT/ART priming without needing the resulting bytes.
<code>encodeToSink(sink, value)</code>	Alias for <code>serialize(sink, value)</code> .
<code>encodeToSink(sink, value, clazz)</code>	Non-inline variant of <code>encodeToSink</code> for contexts where the type is only known as a KClass at runtime (e.g. Spring, Retrofit).

Deserialization

Method	Description
---	---
<code>deserialize<T>(json)</code>	Deserializes a JSON String into an instance of type T.
<code>deserialize<T>(source)</code>	Deserializes from an Okio BufferedSource stream. Reads all bytes eagerly into a reusable scratch buffer.
<code>deserialize<T>(bytes)</code>	Deserializes a UTF-8 JSON ByteArray into an instance of type T.
<code>deserialize<T>(json, options)</code>	Advanced: Deserializes a JSON String with custom parser settings.
<code>deserialize<T>(source, options)</code>	Advanced: Deserializes from a BufferedSource with custom parser settings.
<code>deserialize<T>(bytes, options)</code>	Advanced: Deserializes a ByteArray with custom parser settings.
<code>decodeFromBytes(bytes, clazz, limit)</code>	Non-inline variant for reflection/framework contexts (Spring, Retrofit).

decodeFromSource(source, clazz)**Non-inline variant of deserialize for BufferedSource.**

Registry management

Method	Description
;---	;---
addRegistry(registry)	Registers a GhostRegistry manually. Optional on iOS and Wasm when the Ghost Gradle plugin links serializers (default); required there with <code>autoRegistration = false</code> , and for name-based lookups or <code>prewarm()</code> .
getSerializer(clazz)	Resolves the GhostSerializer for a given class. Checks primitives first, then the fast-path cache, then registered modules, then (Native/Wasm) the compiler-plugin link.
getSerializer(type)	Resolves the GhostSerializer for a KType, supporting generic types (<code>List<T></code> , <code>Map<K,V></code>).
prewarm()	Triggers eager loading and JIT/ART warm-up cycles for all registered serializers. Call at app startup for zero-latency first-run deserialization.
throwError(message)	Throws <code>IllegalArgumentException</code> . Utility for generated serializers.

A.2 GhostRegistry — Module contract

interface GhostRegistry in package `com.ghost.serialization.contract`.

Registry interface for discovering and managing compiler-generated and custom serializers.

Implementations are typically generated automatically as module-level registries by the Ghost

compiler plugin, allowing reflection-free serializer lookup across all targets in a Kotlin

Multiplatform project.

Method	Description
;---	;---
getSerializer(clazz)	Resolves a GhostSerializer for the given class. Returns null if not registered in this module.
getAllSerializers()	Returns all serializers registered in this module. Used by Ghost for eager loading and <code>prewarm</code> .
prewarm()	Eagerly initializes registry entries. No-op by default; can be overridden.
registeredCount()	Returns the total number of serializers in this registry.

A.3 GhostJsonException — Parsing exception

class `GhostJsonException` in package `com.ghost.serialization.exception`.
Exception type thrown for JSON parsing/encoding errors.
To keep the failure path cheap (the parser may raise this exception in tight loops while probing payloads), line and column are computed lazily — the $O(N)$ scan over the source bytes is only paid if the caller actually reads either property or accesses message.

Property / Constructor	Description
<code>---</code>	<code>---</code>
<code>path: String</code>	The dot-separated JSON path where the error occurred. Defaults to "\$" (root).
<code>line: Int</code>	The 1-indexed line number in the JSON source where the error occurred. Computed lazily.
<code>column: Int</code>	The 1-indexed column number in the JSON source where the error occurred. Computed lazily.
<code>message</code>	Full message: "<msg> [at line X, col Y, path Z]".
<code>constructor(message, line, column, path)</code>	Secondary constructor with an explicit error location.

A.4 `JsonReaderOptions` — Optimized field dispatch

class `JsonReaderOptions` in package `com.ghost.serialization.parser`.
Dispatch options for optimized JSON field identification. Uses a 4-byte hashing engine to minimize collisions during field lookup.
`rawBytes` stores field names as raw `ByteArray` instead of Okio's `ByteString`,
so that `verifyKeyMatch` can compare bytes directly without virtual dispatch
or redundant bounds checks inside Okio's `rangeEquals`.

Factory	Description
<code>---</code>	<code>---</code>
<code>JsonReaderOptions.of(vararg names)</code>	Creates an optimized configuration with default hashing parameters (<code>shift=0</code> , <code>multiplier=31</code>).
<code>JsonReaderOptions.of(shift, multiplier, vararg names)</code>	Creates a configuration with custom hashing shift and multiplier values to minimize collisions for a specific field set.

A.5 `@InternalGhostApi` — Internal use annotation

annotation class `InternalGhostApi` in package `com.ghost.serialization`.

Marks declarations that are internal to Ghost Serialization.

Declarations annotated with this annotation are not intended for public use and may

change or be removed in future versions without notice. Code generated by the Ghost KSP

compiler plugin uses this annotation to access internal optimized helper functions.

Opt-in level: `RequiresOptIn.Level.WARNING`. User code that directly uses this API will receive a compiler warning.
